一文搞懂大模型在多GPU环境的分布式训练！

Original 郭小喵玩AI 小喵学AI 2024年07月30日 20:00

小喵学AI
专注于分享C++/Python编程、计算机视觉、自然语言处理、大模型等深度学习与人工智能...
139篇原创内容

公众号

随着大模型时代的到来，模型参数量、训练数据量、计算量等各方面急剧增长。大模型训练面临新的挑战：

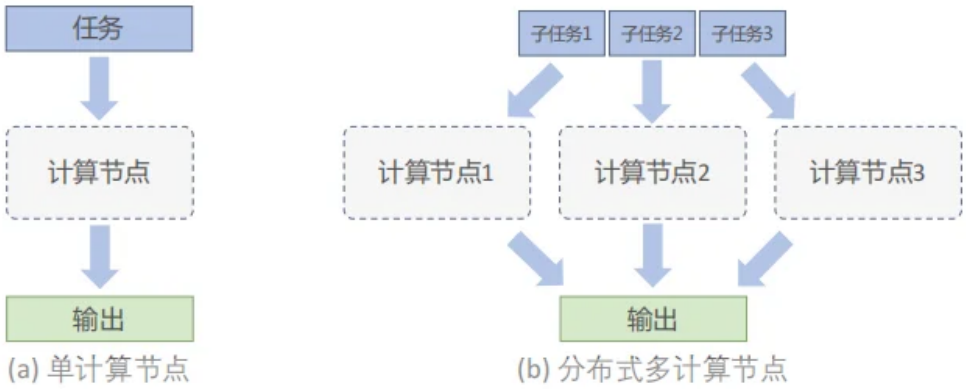
1. 显存挑战：例如，175B的GPT-3模型需要175B*4bytes即700GB模型参数空间，而常见的GPU显存如A100是80G显存，这样看来连模型加载都困难更别说训练。
2. 计算挑战：175B的GPT-3模型计算量也很庞大了，再叠加预训练数据量，所需的计算量与BERT时代完全不可同日而语。

分布式训练(Distributed Training)则可以解决海量计算和内存资源要求的问题。它可将一个模型训练任务拆分为多个子任务，并将子任务分发给多个计算设备（eg:单机多卡，多机多卡），从而解决资源瓶颈。

本文将详细介绍分布式训练的基本概念、集群架构、并行策略等，以及如何在集群上训练大语言模型。

何为分布式训练？

分布式训练是指将机器学习或深度学习模型训练任务分解成多个子任务，并在多个计算设备上并行训练，可以更快速地完成整体计算，并最终实现对整个计算过程的加速。



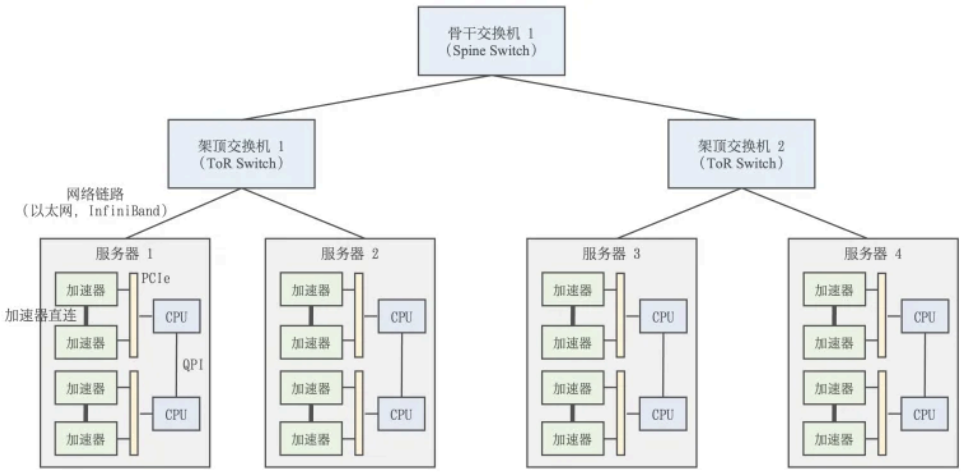
如上图是单个计算设备和多个计算设备的不同，这里计算设备可以是CPU、GPU、TPU、NPU等。

在分布式训练的背景下，无论是单服务器内的多计算设备还是跨服务器的多设备，系统架构均被视为**「分布式系统」**。这是因为，即使在同一服务器内部，多个计算设备(如GPU)之间的内存也不一定是共享的，意味着**「设备间的数据交换和同步必须通过网络或高速互联实现」**，与跨服务器的设备通信本质相同。

分布式训练集群架构

分布式训练集群属于高性能计算集群（High Performance Computing Cluster，HPC），其目标是提供海量的计算能力。 在由高速网络组成的高性能计算上构建分布式训练系统。

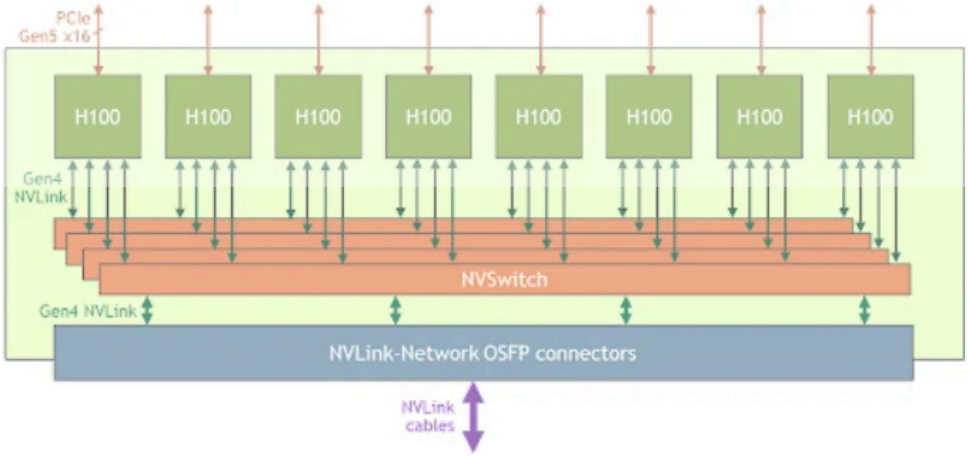
高性能计算集群硬件组成如图所示。



整个计算集群包含大量带有计算加速设备的服务器，多个服务器会被安置在机柜中，服务器通过架顶交换机（Top of Rack Switch，ToR）连接网络。在架顶交换机满载的情况下，可以通过在架顶交换机间增加骨干交换机进一步接入新的机柜。

每个服务器中通常是由2-16个计算加速设备组成，这些计算加速设备之间的高速通信直接影响到分布式训练的效率。传统的PCI Express（PCIe）总线，即使是PCIe 5.0版本，也只能提供相对较低的128GB/s带宽，这在处理大规模数据集和复杂模型时可能成为瓶颈。

为了解决这一问题，NVIDIA推出了NVLink和NVSwitch技术。如下图所示，每个H100 GPU都有多个NVLink端口，并连接到所有四个NVSwitch上。每个NVSwitch都是一个完全无阻塞的交换机，完全连接所有8个H100计算加速卡。NVSwitch的这种完全连接的拓扑结构，使得服务器内任何H100加速卡之间都可以达到900GB/s双向通信速度。



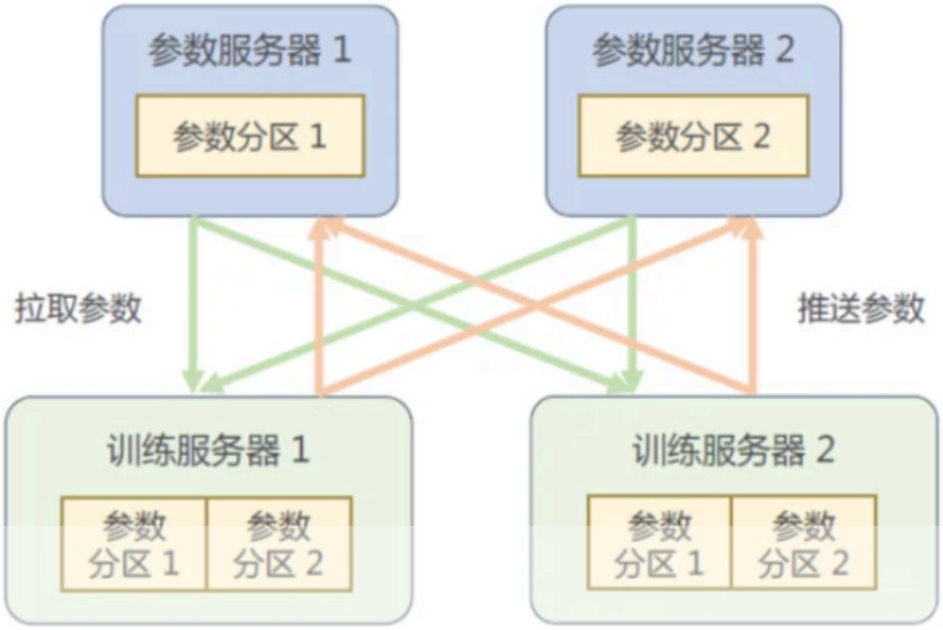
针对分布式训练服务器集群进行架构涉及的主流架构，目前主流的主要分为参数服务器（ParameterServer，简称PS）和去中心化架构（Decentralized Network）两种分布式架构。

参数服务器架构

参数服务器架构的分布式训练系统中有两种服务器：

- 训练服务器：提供大量的计算资源
- 参数服务器：提供充足的内存资源和通信资源

如下所示是具有参数服务器的分布式训练集群的示意图。在训练过程中，每个训练服务器都拥有完整的模型，并根据将分配到此服务器的训练数据集切片（Dataset Shard）进行计算，将得到的梯度推送到相应的参数服务器。参数服务器会等待两个训练服务器都完成梯度推送，然后开始计算平均梯度，并更新参数。之后，参数服务器会通知训练服务器拉取最新的参数，并开始下一轮训练迭代。



去中心化架构

去中心化架构没有中央服务器或控制节点，而是由节点之间进行直接通信和协调，这种架构的好处是可以减少通信瓶颈，提高系统的可扩展性。

节点之间的分布式通信一般有两大类：

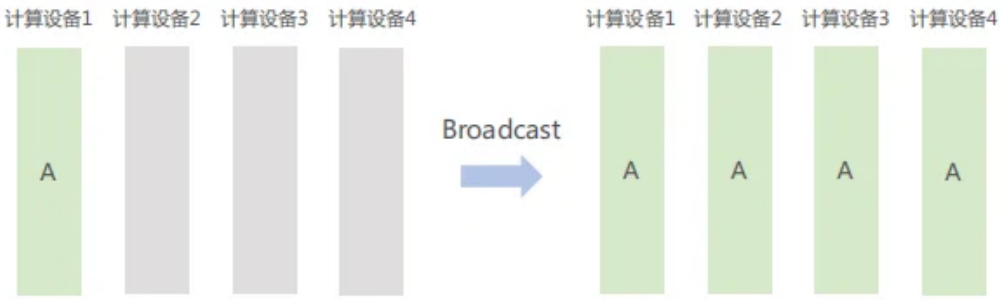
- 点对点通信（Point-to-Point Communication）：在一组节点内进行通信
- 集合通信（Collective communication，CC）：在两个节点之间进行通信

去中心化架构中通常采用集合通信实现。

常见通信原语如下：

「Broadcast」

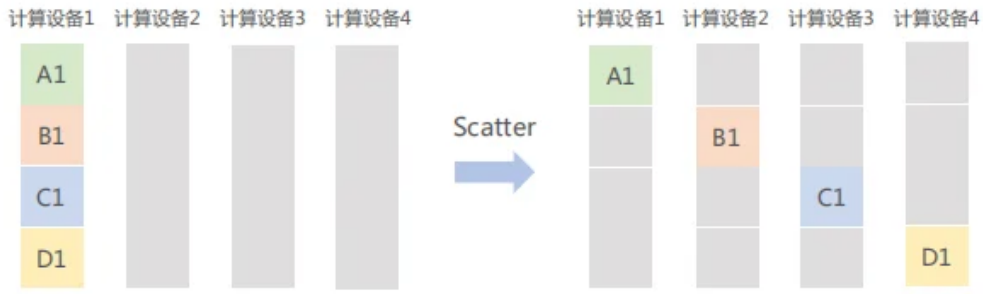
将数据从主节点发送到集群中的其他节点。如下图，计算设备1将大小为1xN的张量广播到其它设备，最终每张卡输出均为1×N矩阵。



Broadcast在分布式训练中主要用于**「模型参数初始化」**。

「Scatter」

主节点将一个大的数据块分割成若干小部分，再将每部分分发到集群中的其他节点。如下图，计算设备1将大小为1xN的张量分成4个子张量，再分别发送给其它设备。

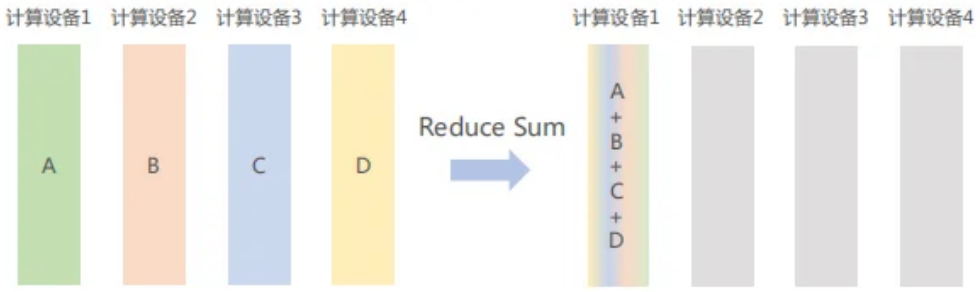


「Reduce」

将不同节点上的计算结果进行聚合。Reduce操作可以细分为多种类型，包括SUM（求和）、MIN（求最小值）、MAX（求最大值）、PROD（乘积）、LOR（逻辑或）

等，每种类型对应一种特定的聚合方式。

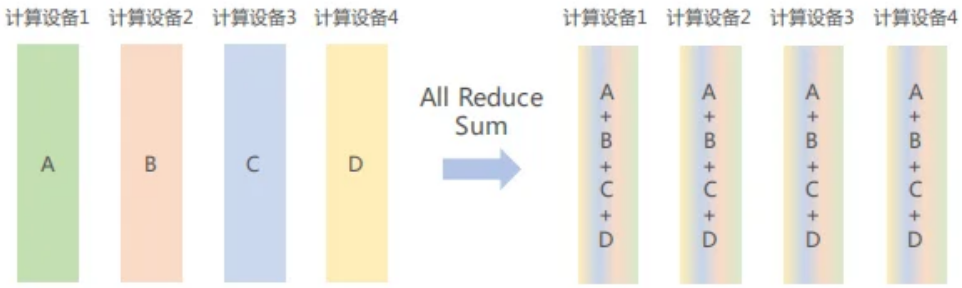
如下图所示，Reduce Sum操作将所有计算设备上的数据进行求和，然后将结果返回到计算设备1。



「All Reduce」

在所有节点上执行同样的Reduce操作，如求和、求最小值、求最大值等。可通过单节点上Reduce+Broadcast操作完成。

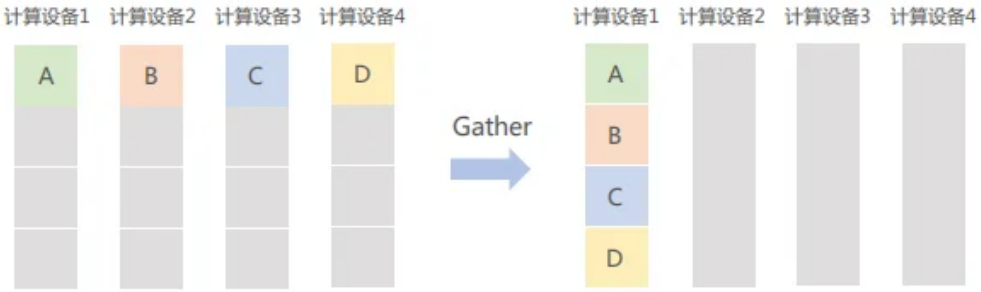
如下图所示，All Reduce Sum操作将所有节点上的数据求和，然后将求和结果Broadcast到所有节点。



「Gather」

将所有节点的数据收集到单个节点，可以看作是Scatter操作的逆操作。

如下图所示，Gather操作将所有设备的数据收集到计算设备1中。

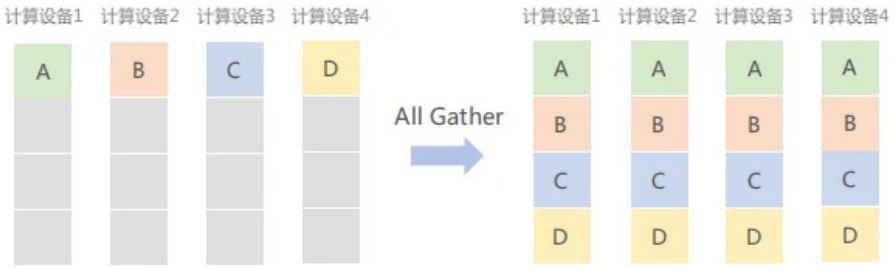


- 「All Gather」

在所有节点上收集所有其他节点的数据，最终使每个节点都拥有一份完整的数据

集合。可以视为Gather操作与Broadcast操作的结合体。

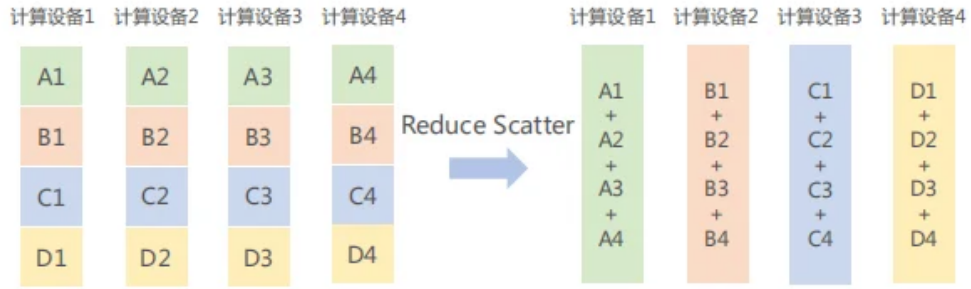
如下图所示，All Gather操作将所有计算设备上的数据收集到各个计算设备。



- 「Reduce Scatter」

  将每个节点的张量分割成多个块，每个块分发给不同的节点，再在每个节点执行Reduce操作（如求和、平均等）。

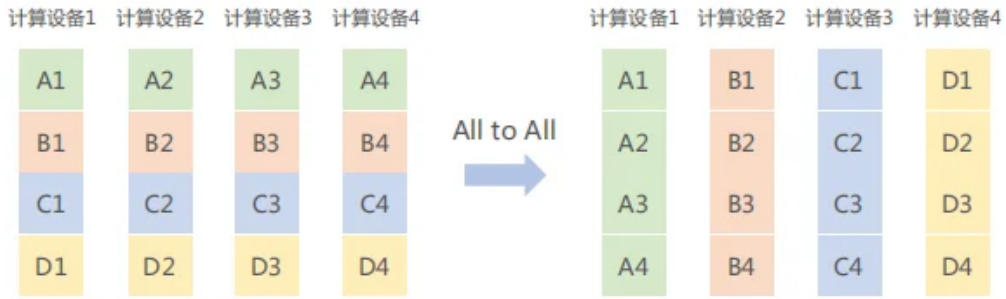
  如下图所示，Reduce Scatter操作将每个计算设备中的张量分割成4块，并发送给4个不同的计算设备，每个计算设备对接收到的块执行Reduce Sum操作。



「All to All」

将每个节点上的数据分割成多个块，并将这些块分别发送给不同的节点。

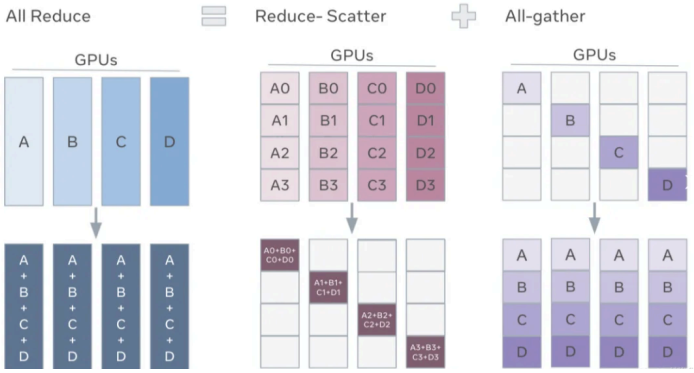
如下图所示，All to All操作将每个计算设备中的张量分割成4块，并发送给4个不同的计算设备。



分布式训练并行策略

分布式训练系统的核心目标是将原本在单一计算节点上进行的模型训练过程，转化为能在多个计算节点上并行执行，以加速训练速度并支持更大规模的模型和数据集。

在单节点模型训练中，系统结构主要由两大部分组成：数据和模型。训练过程由多个数据小批次（Mini-batch）完成。如图所示，数据表示一个数据小批次。训练系统会利用数据小批次根据损失函数和优化算法生成梯度，从而对模型参数进行修正。



针对大语言模型多层神经网络的执行过程，模型训练过程可以抽象为一个计算图（Computational Graph）。这个图由多个相互连接的算子（Operator）构成，每个算子对应神经网络中的一个层（Neural Network Layer），如卷积层、全连接层等。参数（Weights）则是这些层在训练过程中不断更新的权重。

计算图的执行过程可以分为前向传播和反向传播两个阶段。

「前向计算（Forward Pass）」

1. 输入数据：数据从输入层开始，被送入计算图的第一个算子。
2. 算子执行：每个算子接收输入数据，执行相应的数学运算（如矩阵乘法、激活函数等），并产生输出。
3. 数据传递：算子的输出作为后续算子的输入，沿着计算图向前传播。
4. 输出生成：当数据到达计算图的末端，即输出层，产生最终的预测结果。

「反向计算（Backward Pass）」

1. 损失计算：在前向传播完成后，使用损失函数比较预测输出与实际标签，计算损失值。
2. 梯度计算：从输出层开始，反向遍历计算图，根据损失值和算子的导数，计算每个算子的梯度。
3. 参数更新：利用计算出的梯度，根据选择的优化算法（如梯度下降、Adam等），更新模型参数。
4. 传播回溯：反向计算过程从输出层向输入层递归进行，直到所有参数都被更新。

根据单设备模型训练流程，可以看出，如果进行并行加速，可以从数据和模型两个维度考虑：

1. 对数据进行切分（Partition），并将同一个模型copy到多个设备上，每个设备并行执行不同的数据分片，即**「数据并行（Data Parallelism，DP）」**。
2. 对模型进行拆分，将模型中的算子分发到多个设备分别完成，即**「模型并行（Model Parallelism，MP）」**。
3. 训练超大规模语言模型时，同时对数据和模型进行并行，即**「混合并行（Hybrid Parallelism，HP）」**。

数据并行DP

数据并行是最常用的并行训练方式，主要分为DataParallel(DP)和DistributedDataParallel(DDP)两种。

「DP」

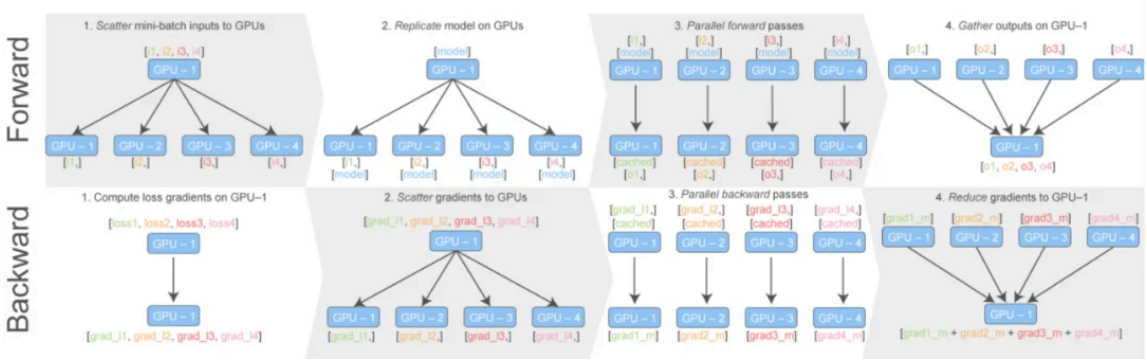
DP是早使用的数据并行方案，通过torch.nn.DataParallel()来调用，代码如下:

```
# 设置可见的GPU
os.environ['CUDA_VISIBLE_DEVICES'] = "0,1,2,3"

# 将模型放到GPU 0上，必须先把模型放在GPU上，后面才可以调用DP
model.cuda()

# 构建DataParallel数据并行化
model=torch.nn.DataParallel(model)
```

DP核心思想是将一个大的batch数据分割成多个子batch，并将子batch分配给不同的GPU进行并行计算。如下图将训练过程分为前向传播和反向传播详细分析：



「前向传播：」

1. 模型和完整的mini-batch数据被放置在Master GPU（例如GPU:0）上。
2. GPU:0将mini-batch数据分割成若干个子batch，并将这些子batch分发（scatter）到其它GPU上。

3. GPU:0将自身的模型参数复制到其它GPU，确保每个GPU上的模型参数完全相同。
4. 每个GPU在单独的线程上对其sub-mini-batch的数据前向传播，计算出各自的输出结果。
5. GPU:0收集所有GPU的输出结果。

”

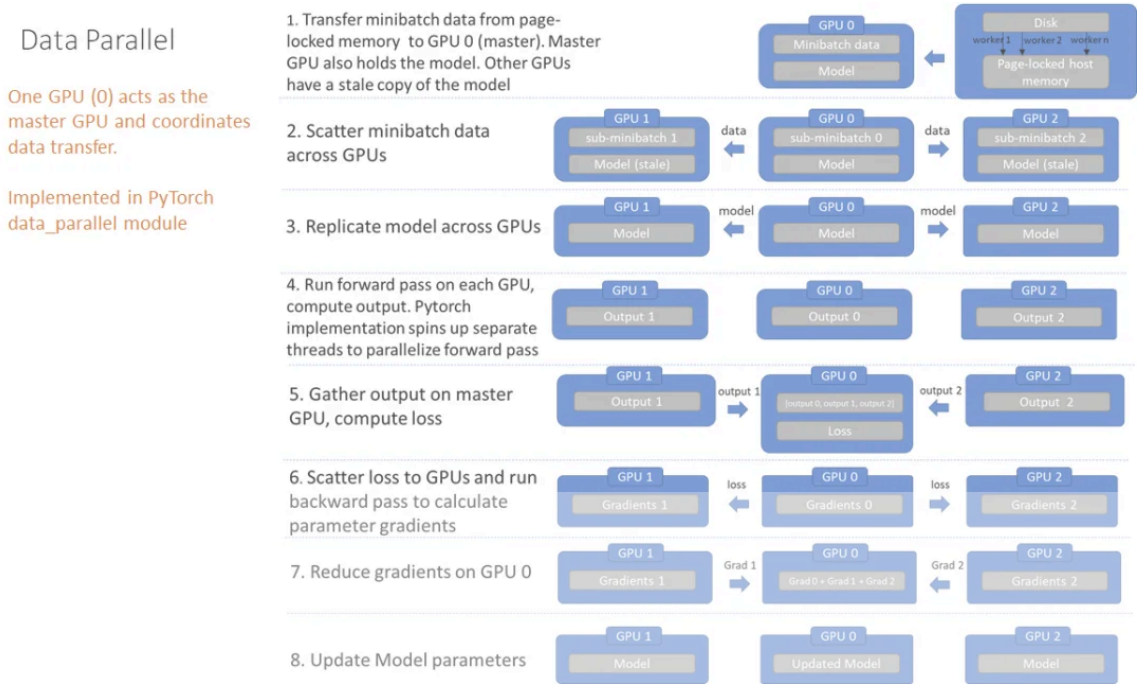
「反向传播：」

1. GPU:0基于收集的输出结果和真实label计算总损失loss，并得到loss的梯度。
2. GPU:0将计算出的loss梯度分发（scatter）到所有GPU上。
3. 每个GPU根据收到的loss梯度反向传播，计算出所有模型参数的梯度。
4. 所有GPU计算出的参数梯度被汇总回GPU:0。
5. GPU:0基于汇总的梯度更新模型参数，完成一次迭代的训练。

”

有人说GPU:0好自私，把其它GPU当做工具人，核心机密不对外，只给其他GPU数据，不给label，其它GPU得到结果它再给计算loss和loss梯度，然后分发给其他GPU去计算参数梯度，之后得到这些参数的梯度后再去更新参数，等下次需要其它GPU了再分发更新好的参数。

这是一个悲伤的故事，首先**「单进程多线程」**就似乎已经注定的结局，Python的全局解释锁给这些附属的GPU戴上了沉沉的牢拷，其他GPU想奋起反抗，但是DP里面只有一个优化器Optimizer，这个优化器Optimizer只在Master GPU上进行参数更新，当环境不再不在改变的时候，其它GPU选择了躺平，当GPU:0忙前忙后去分发数据、汇总梯度，更新参数的时候，其它GPU就静静躺着。



DataParallel采用的是Parameter Server并行架构，在实现多GPU或多节点并行训练时，存在一些固有的局限性：

- 通信开销大：每个**「计算节点」**在每次迭代中都需要与参数服务器进行多次通信，以获取最新的参数更新并将计算的梯度发送回去。这种频繁的通信会导致网络带宽成为瓶颈，尤其是当模型参数量大且GPU数量众多时，通信延迟和带宽消耗会显著影响整体训练速度。
- 负载不均衡：其中一个GPU被指定为Master GPU，负责汇总梯度和广播更新等，Master GPU可能会承担额外的通信和计算负担，导致负载不均衡。这不仅会影响该GPU的计算效率，也可能拖慢整个训练过程的速度。同时导致GPU利用率不足。
- **「仅支持单机多卡模式，无法实现多机多卡训练。」**

「DistributedDataParallel（DDP)」

DDP采用多进程架构，赋予了每个GPU更多的自由，支持多机多卡分布式训练。每个进程都拥有自己的优化器Optimizer，可独立优化所有步骤。每个进程中在自己的GPU上计算loss，反向计算梯度。

在DDP中，不存在所谓的Master GPU，所有GPU节点地位平等，共同参与训练过程的每一个环节。每个GPU都会计算loss和梯度，然后通过高效的通信协议（如AllReduce）与其它GPU同步梯度，确保模型参数的一致性。

实现代码如下：

```
# 初始化分布式环境
-------------------------------------------------------------------------
# 1) 指定通信后端为nccl（NVIDIA Collective Communications Library），
  # 这是针对GPU集群优化的高性能通信库
 torch.distributed.init_process_group(backend='nccl')

# 2）从命令行接收local_rank参数，该参数表示当前GPU在本地机器上的编号，用于后续的设备设
parser = argparse.ArgumentParser()
parser.add_argument("--local_rank", default=-1, type=int)
args = parser.parse_args()

# 3）设置cuda
# 根据local_rank设置当前进程使用的GPU设备，创建对应的device对象
torch.cuda.set_device(args.local_rank)
device = torch.device("cuda", args.local_rank)

-------------------------------------------------------------------------
# 模型设置
# 将模型封装进DistributedDataParallel，
# 指定模型运行在local_rank对应的GPU上，同时将模型移动到相应的设备
model = torch.nn.parallel.DistributedDataParallel(model, device_ids=[l
model.to(device)
-------------------------------------------------------------------------
```

```
# Dataset设置
Test_data = FunDataset(args.input)
# 创建DistributedSampler，用于在分布式环境中对数据集进行采样，确保每个进程处理不同的数
test_sample = torch.utils.data.distributed.DistributedSampler(Test_dat
# 使用DataLoader加载数据，指定sampler为DistributedSampler，确保数据的分布式加载和
test_data_dataset = DataLoader(dataset=Test_data, batch_size=args.batc
                                    collate_fn=Test_data.collate__fn,
                                    drop_last=False,sampler=test_sample
-----------------------------------------------------------------------
# 运行的时候需要设置
for epoch in range(num_epochs):
    #  在每个epoch开始时，更新DistributedSampler的epoch，确保数据的随机重排
     trainloader.sampler.set_epoch(epoch)
    #  遍历数据集，前向传播计算预测值，计算损失，执行反向传播和参数更新
     for data, label in trainloader:
        prediction = model(data)
        loss = loss_fn(prediction, label)
        loss.backward()
        optimizer = optim.SGD(ddp_model.parameters(), lr=0.001)
        optimizer.step()
```

DDP解决了DP模式下存在的效率瓶颈和资源分配不均等问题，每个GPU节点都具备独立的数据加载能力，无需等待主GPU的数据分发。并且，可执行模型训练的每一环节，包括前向传播、损失计算和反向传播，实现了真正的并行计算。

引入了DistributedSampler，用于在分布式环境中均匀分割数据集，确保每个GPU处理的数据互不重叠，避免了数据冗余和计算浪费。

采用了高效的Ring All-Reduce算法作为通信后端，用于梯度的聚合和参数的同步。在每个训练迭代结束时，每个GPU计算出自己的梯度后，通过环形网络结构与其他GPU进行梯度交换，最终每个GPU都能获取到所有GPU的梯度信息。

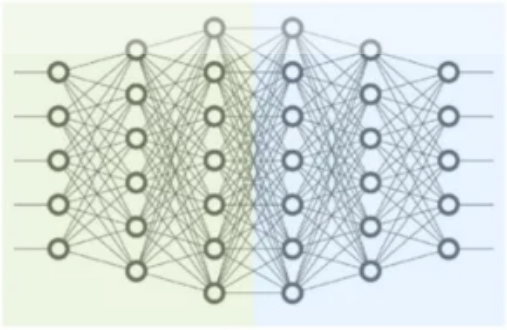
针对DP来说，Dataloder的batch_size是针对所有卡训练batch_size的和，例如10卡，每张卡batch_size是20，那么就要指定batch_size为200。针对DDP来说，batch_size就是每张卡所训练使用的batch_size为20。

模型并行MP

模型并行（Model Parallelism）通常用于解决单节点内存不足的问题。以GPT-3为例，该模型拥有1750亿参数，如果每个参数都使用32位浮点数表示，那么模型需要占用700GB（即175G× 4 Bytes）内存。即使使用16位浮点数表示，每个模型副本也需要350GB的内存。单个加速卡（如NVIDIA H100）的显存容量（80GB）显然不足以容纳整个模型。

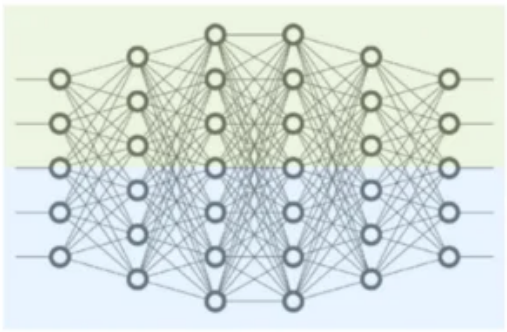
模型并行从计算图的切分角度，可以分为以下两种：

1. 按模型的**「layer层切分」**到不同设备，即**「层间并行或算子间并行」**（Inter-operator Parallelism），也称之为**「流水线并行」**（Pipeline Parallelism，PP）。



层间并行

2. 将计算图层内的**「参数切分」**到不同设备，即**「层内并行或算子内并行」**（Intra-operator Parallelism），也称之为**「张量并行」**（Tensor Parallelism，TP）。



层内并行

张量并行

张量并行（Tensor Parallelism，TP）旨在通过将模型参数和中间张量在多个设备（如GPU）之间进行切分，以克服单个设备内存限制的问题。

张量并行要解决的两个核心问题：如何合理地将参数切分到不同设备，以及如何确保切分后的计算结果在数学上保持一致。

大语言模型都是以Transformer结构为基础，Transformer结构主要由以下三种算子构成：

- 嵌入式表示（Embedding）
- 矩阵乘法（MatMul）
- 交叉熵损失（Cross Entropy Loss）

这三种类型的算子均需要设计对应的张量并行策略，才可以将参数切分到不同设备。

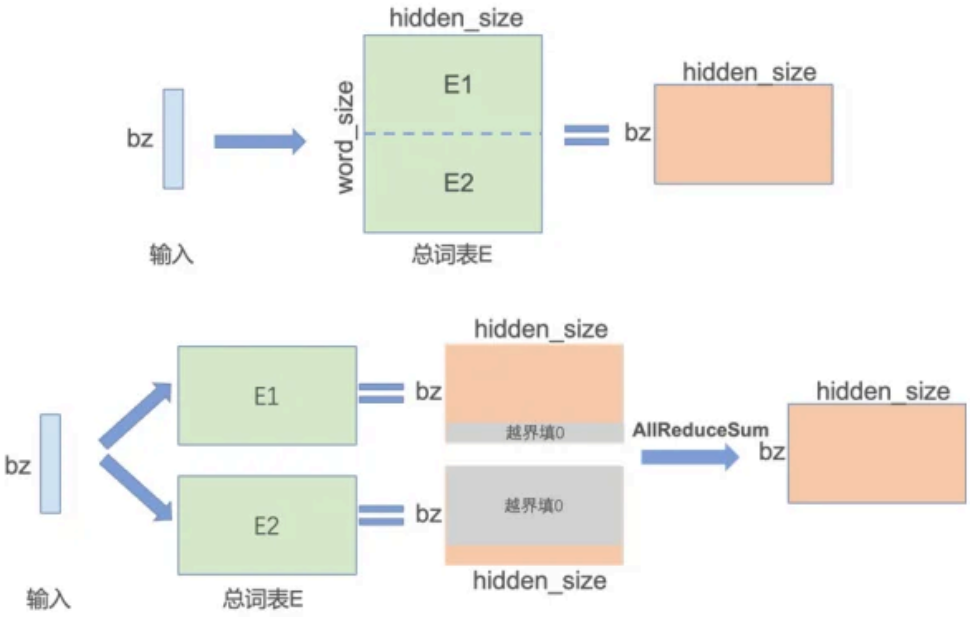
「嵌入式表示（Embedding）」

对于Embedding算子，总词表数很大，将导致单计算设备显存无法容纳Embedding层参数。

例如，词表数为64000，嵌入表示维度为5120，使用32位浮点数表示，整层参数需要的显存大约为6400x5120x4/1024/1024=1250MB，加上反向传播时的梯度存储，总计近2.5GB，这对于显存有限的设备而言是一个严峻挑战。

为了解决这一问题，可以采用张量并行策略，将Embedding层的参数按词维度切分，每个计算设备只存储部分词向量，然后通过汇总各个设备上的部分词向量，从而得到完整的词向量。

如下图所示是单节点Embedding和两节点张量并行的示意图。



在单节点上，执行Embedding操作，bz是批次大小（batch size），Embedding的参数大小为[word_size, hidden_size]，计算得到[bz,hidden_size]张量。

在两节点上，可以将Embedding层参数按词维度切分为两半，每台设备存储一半的词向量，即参数大小为[word_size/2, hidden_size]，分别存储在两个设备上。在前向传播过程中，每个设备根据输入的词汇ID查询自身的词向量。如果无法查到，则该词的表示为0。各设备计算得到的词向量结果形状为[bz, hidden_size]，由于词汇可能被分割在不同设备上，需要通过跨设备的All Reduce Sum操作，将所有设备上的词向量结果进行汇总求和，以得到完整的词向量表示。可以看出，这里的输出结果和单节点执行的结果一致。

「矩阵乘法（MatMul）」

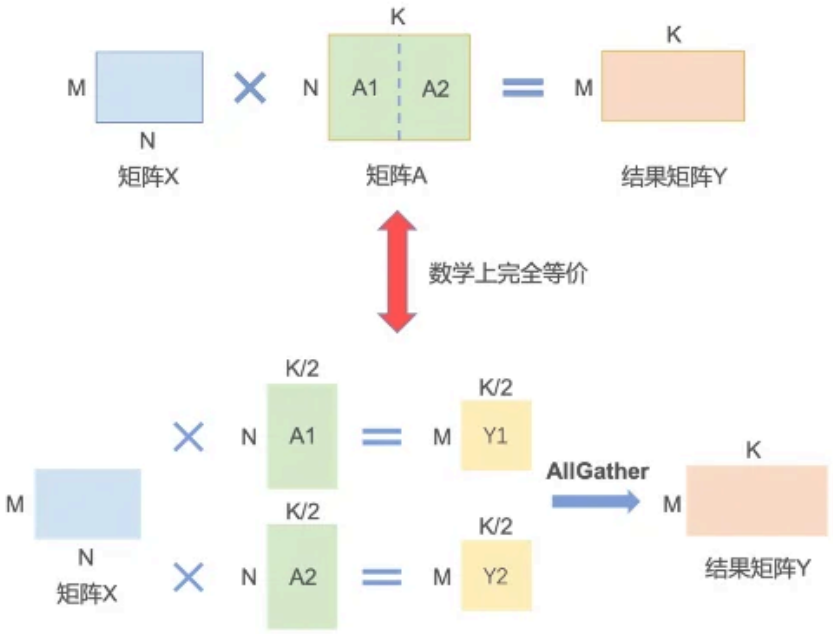
矩阵乘的张量模型并行充分利用矩阵分块乘法的原理。

例如，要实现矩阵乘法Y=X*A。其中，X是维度为MxN的输入矩阵，A是维度为NxK的参数矩阵，Y是输出，维度为MxK。

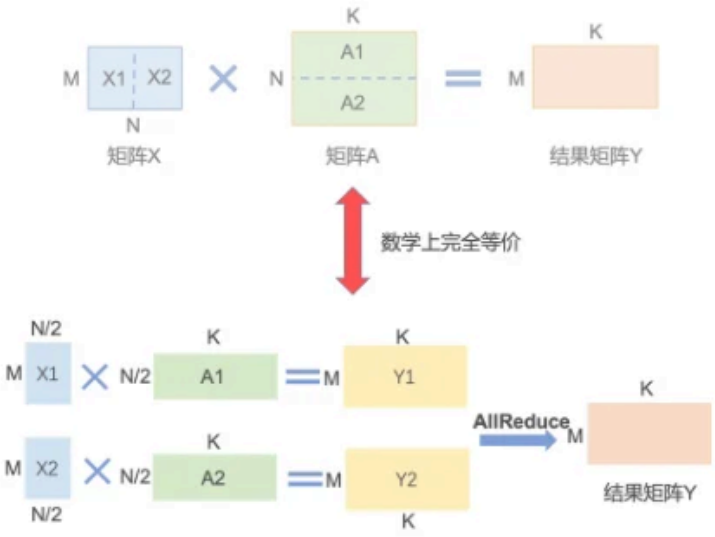
当参数矩阵A的尺寸过大，以至于单个卡无法容纳时，可以将参数矩阵A切分到多张卡上，并通过集合通信汇集结果，保证最终结果在数学计算上等价于单卡计算结果。

这里A切分方式包括按列切块和按行切块两种。

- **「按列切块」**。如下图所示，将参数矩阵A按列方向切分为A1和A2。将子矩阵A1和A2分配到两张卡上。在计算过程中，每张卡将执行独立的矩阵乘法，即卡1计算Y1=X*A1，卡2计算Y2=X*A2。计算完成后，通过All Gather操作，每张卡将获取另一张卡上的计算结果。在收集到所有计算结果后，每张卡将拼接它们收到的片段，形成完整的Y矩阵。最终，无论在哪张卡上查看，Y都将是一个完整的MxK矩阵，与单卡计算的结果完全等价。



- **「按行切块」**。如下图所示，将参数矩阵A按列方向切分为A1和A2。为了与按行切块的A矩阵相乘，输入矩阵X（尺寸为MxN）也需要按列方向切分为X1和X2。将子矩阵A1和A2分别分配到两张卡上。同时，将X1和X2也分别分配到对应的卡上。每张卡将执行独立的矩阵乘法，即卡1计算Y1=X1*A1，卡2计算Y2=X2*A2。计算完成后，通过All Reduce Sum操作，每张卡将汇总另一张卡上的计算结果。在收集到所有计算结果后，每张卡将整合它们收到的片段，形成完整的Y矩阵的行。同理，参数矩阵A按行切块的张量模型并行策略，通过巧妙地切分矩阵和利用多卡的计算能力，有效地解决了单卡显存限制的问题，同时确保了计算结果的数学等价性。



「交叉熵Loss计算（CrossEntropyLoss）」

分类网络中，最后一层通常使用softmax函数结合交叉熵损失（CrossEntropyLoss）来评估模型的预测结果与真实标签之间的差距。然而，当类别数量非常大时，单个GPU卡可能无法存储和计算logit矩阵，导致显存溢出。为了解决这一问题，可以采用张量并行策略，将logit矩阵和标签按类别数维度切分，通过中间结果的通信，计算出全局的交叉熵损失。具体的计算步骤如下：

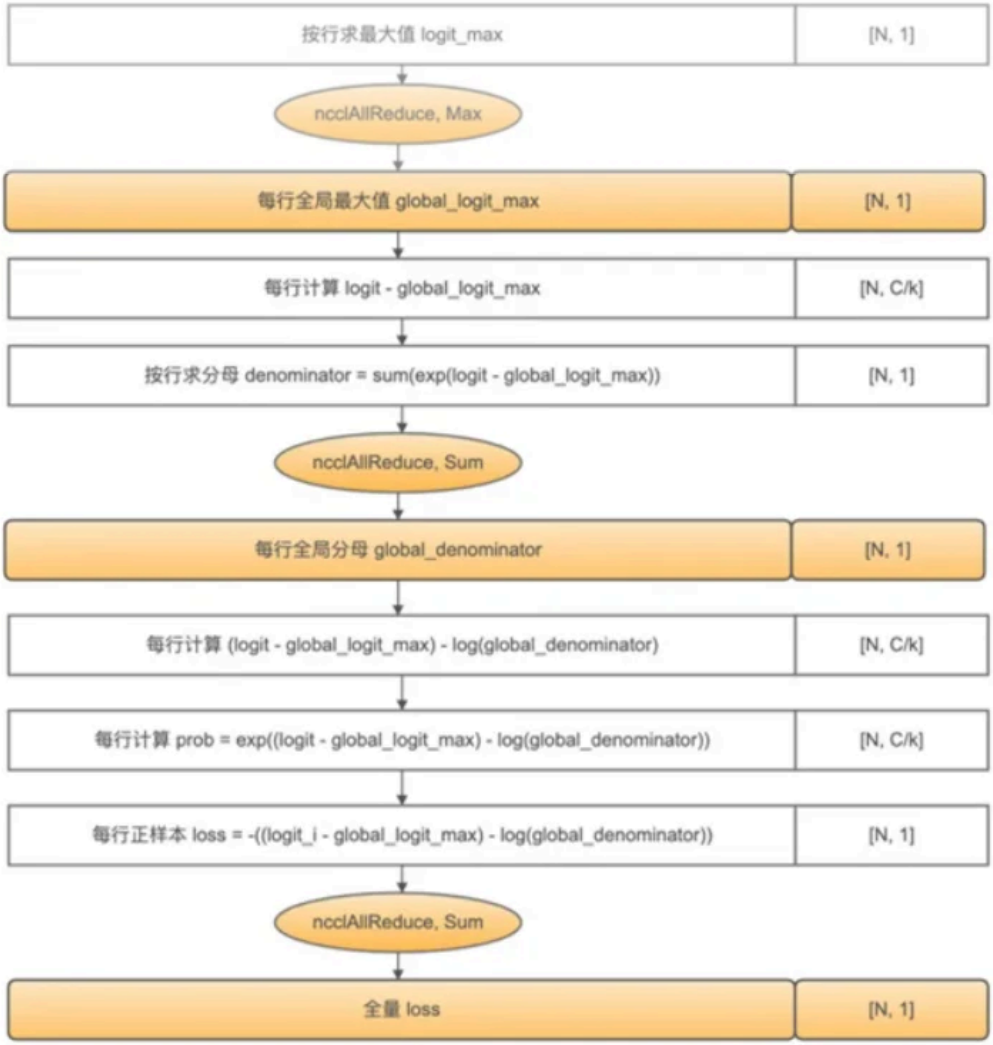
首先计算softmax值。其中，p表示张量并行的设备号。

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\Sigma_j e^{x_j}} = \frac{e^{x_i - x_{max}}}{\Sigma_j e^{x_j - x_{max}}} = \frac{e^{x_i - x_{max}}}{\Sigma_N \Sigma_j e^{x_j - x_{max}}}$$

$$x_{max} = \max_p(\max_k(x_k))$$

得到Softmax结果之后，同时对标签Target按类别切分，每个设备得到部分损失，最后再进行一次通信，得到所有类别的损失。

具体的计算步骤如下图所示。



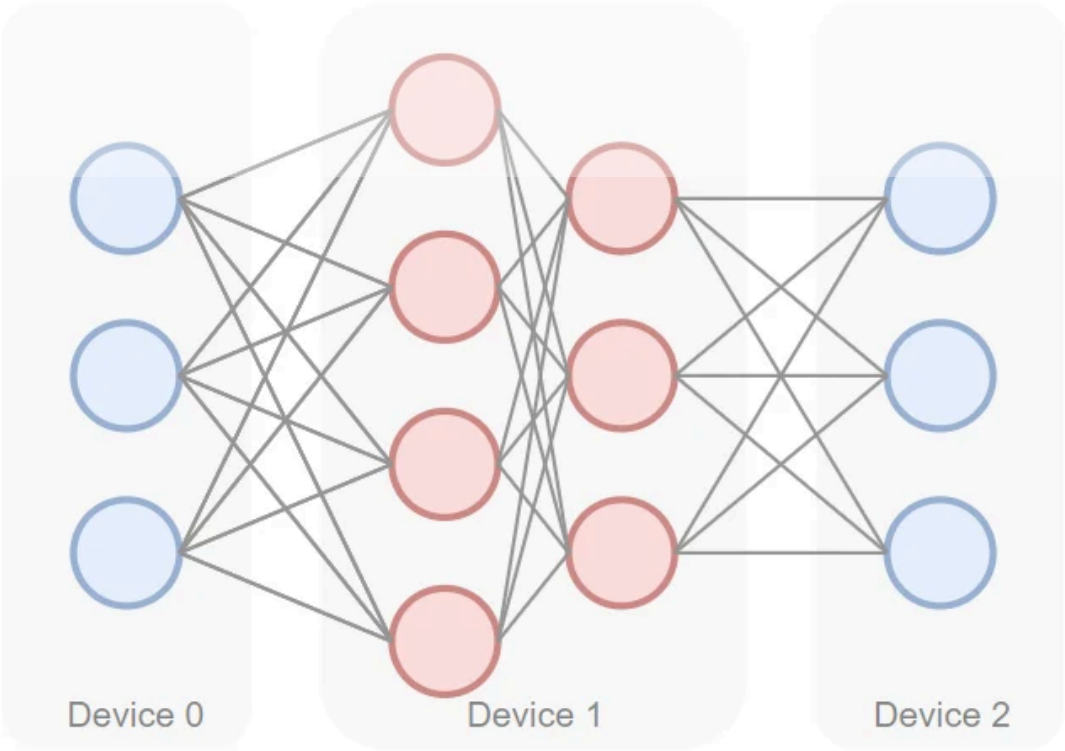
流水线并行

所谓流水线并行，就是由于模型太大，无法将整个模型放置到单张GPU卡中，因此，将模型的不同层放置到不同的计算设备，降低单个计算设备的显存消耗，从而实现超大规模模型训练。

流水线并行PP（Pipeline Parallelism），是一种最常用的并行方式，将模型的各个层分段处理，并将每个段分布在不同的计算设备上，使得前后阶段能够流水式、分批进行工作。

如下图所示是一个包含四层的深度学习模型，被切分为三个部分，并分别部署在三个不同的计算设备（Device 0、Device 1 和 Device 2）上。

其中，第一层（Layer 1）放置在Device 0上。第二层（Layer 2）和第三层（Layer 3）放置在Device 1上。第四层（Layer 4）放置在Device 2上。



前向计算过程中，输入数据首先在 Device 0 上通过Layer 1的计算得到中间结果，并将中间结果传输到Device 1，再继续在Device 1上计算得到Layer 2 和Layer 3的输出，并将模型Layer 3的输出结果传输到Device 2。在 Device 2上，数据经过Layer 4 的计算，得到最终的前向计算结果。反向传播过程类似。

最后，各个设备上的网络层会使用反向传播过程计算得到的梯度更新参数。由于各个设备间传输的仅是相邻设备间的输出张量，而不是梯度信息，因此通信量较小。

通过将模型切分为多个部分并分布到不同的计算设备上，流水线并行策略有效地扩展了可用于训练的GPU显存容量。这意味着原本无法在单一GPU上装载的大模型，现在可以通过类似流水线的方式，利用更多GPU的显存来承载训练中的模型参数、梯度、优化器状态以及激活值等数据，从而实现超大规模模型的高效训练。

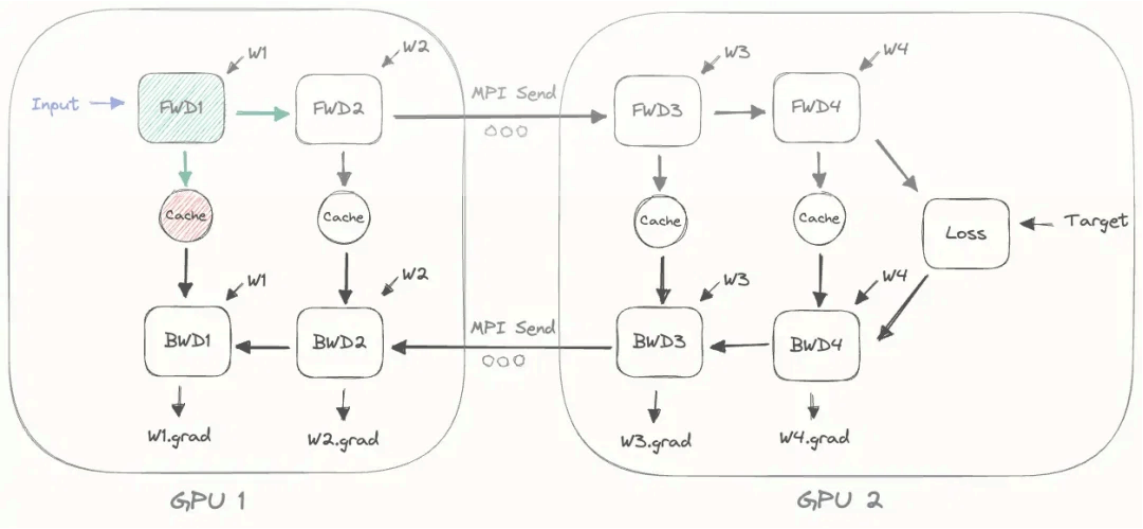
「朴素流水线并行」

当模型规模超过单个GPU的处理能力时，朴素层并行（Naive Layer Parallelism）是一种直观的并行策略，将模型的不同层分配到不同的GPU上，实现模型的并行化训练。如下所示是一个4层序列模型：

```
output=L4(L3(L2(L1(input))))
```

将其划分到两个GPU上：

- GPU1负责计算前两层：intermediate=L2(L1(input))
- GPU2负责计算后两层：output=L4(L3(intermediate))



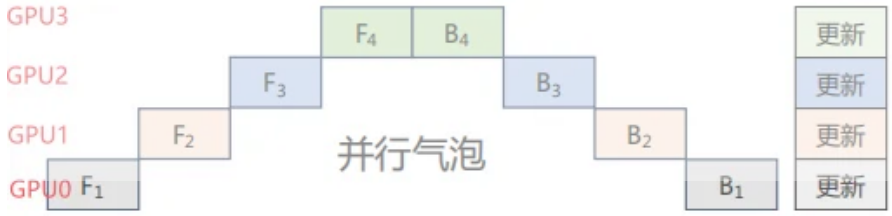
整个朴素层并行前向传播和后向传播的过程如上图所示。GPU1执行Layer 1和Layer 2的前向传播，并缓存激活（activations），再将Layer 2的输出（intermediate）发送给GPU2。GPU2接收来自GPU1的中间结果，执行Layer 3和Layer 4的前向传播，然后计算损失后，开始反向传播，计算Layer 4和Layer 3的梯度，并将Layer 3的梯度返回给GPU1。GPU1接收梯度，继续完成Layer 2和Layer 1的反向传播。

> **「朴素层并行的缺点：」**
>
> - 低GPU利用率：同一时刻，只有其中一个GPU在执行计算，其余GPU处于空闲状态(又称气泡bubble)，这导致了计算资源的浪费。
> - 计算和通信没有重叠：在数据传输期间，无论是前向传播的中间结果还是反向传播的梯度，GPU都处于等待状态，这进一步降低了计算效率。
> - 高显存占用：GPU1需要保存整个mini-batch的所有激活，直到反向传播完成。如果mini-batch很大，这将显著增加显存的需求，可能超出单个GPU的显存容量。

「GPipe流水线并行」

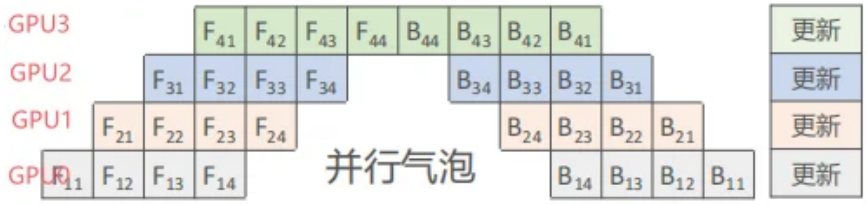
GPipe通过引入微批次（Micro-batch）的概念，显著提高了模型并行训练的效率。将一个大的mini-batch进一步细分为多个更小的、相等大小的微批次（microbatches），并在流水线并行的框架下独立地对每个microbatch执行前向传播和反向传播。然后将每个mircobatch的梯度相加，就能得到整个batch的梯度。由于每个层仅在一个GPU上，对mircobatch的梯度求和仅需要在本地进行即可，不需要通信。

假设我们有4个GPU，并将模型按层划分为4个部分，每个部分部署在一个GPU上。朴素层并行的过程如下所示：



由此可以看出，在每一时刻，仅有一个1个GPU工作，并且每个timesep花费的时间也比较长，因为GPU需要跑完整个minibatch的前向传播。

GPipe将minibatch划分为4个microbatch，然后依次送入GPU0。GPU0前向传播后，再将结果送入GPU1，以此类推。整个过程如下所示。GPU0的前向计算被拆解为F11、F12、F13、F14，在GPU0中计算完成F11后，会在GPU1中开始计算F21，同时GPU0并行计算F12。

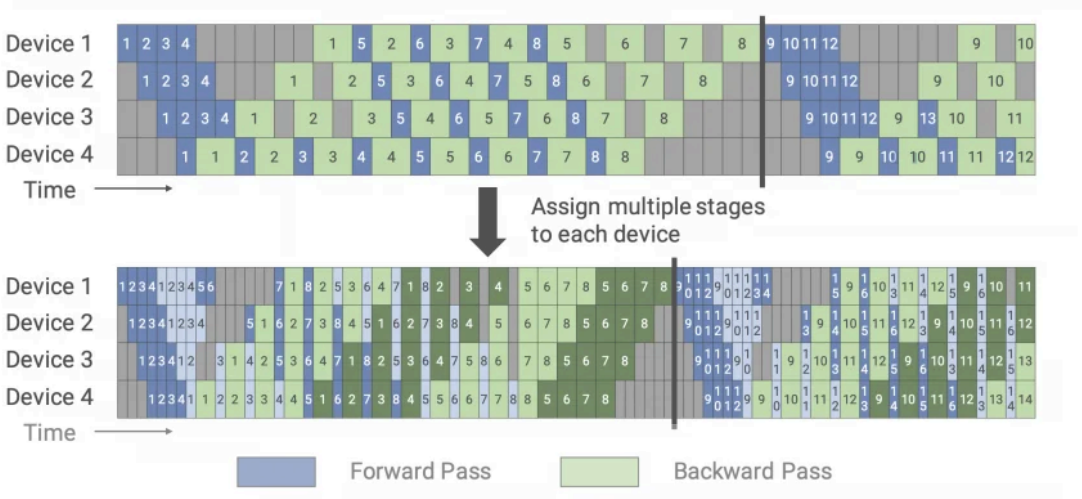


相比于朴素层并行方法，GPipe流水线方法可以有效降低并行气泡大小。但是GPipe只有当一个Mini-batch(4个Microbatch)中所有的前向传播计算完成后，才能开始执行反向传播计算。因此还是会产生很多并行气泡，从而降低了系统的并行效率。每个GPU需要缓存4份中间激活值。

「PipeDream流水线并行」

PipeDream流水线并行采用1F1B策略，即一个前向通道和一个后向通道，采用任务调度机制，使得下游设备能够在等待上游计算的同时执行其他可并行的任务，从而提高设备的利用率。

1F1B策略分为非交错式和交错式两种方式调度方式。如下图所示。



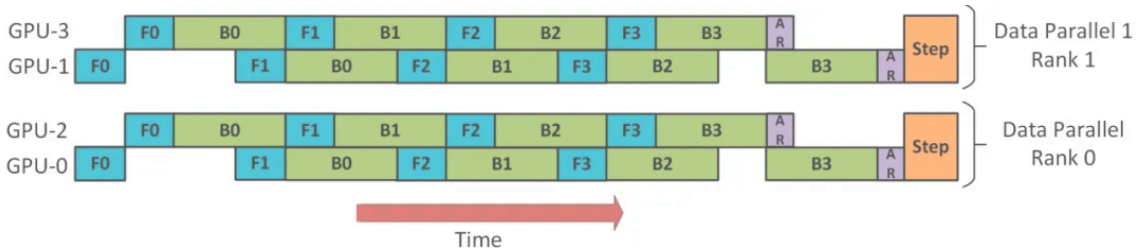
> 「1F1B非交错式调度」分为三个阶段：
>
> 1. 热身阶段：在这个阶段，计算设备执行不同数量的前向计算，为后续阶段做准备。
> 2. 前向-后向阶段：设备按照顺序执行一次前向计算，紧接着进行一次后向计算。这一阶段循环执行，直到所有microbatch被处理完毕。
> 3. 后向阶段：在完成所有前向计算后，设备执行剩余的后向计算，直至所有计算任务完成。

> 「1F1B交错式调度」要求microbatch的数量是流水线阶段的整数倍。 每个设备不再仅负责连续多个层的计算，而是可以处理多个层的子集，这些子集被称为模型块。例如：
>
> - 在传统模式下，设备1可能负责层1-4，设备2负责层5-8。
> - 在交错式模式下，设备1可以处理层1、2、9、10，设备2处理层3、4、11、12，以此类推。 这样，每个设备在流水线中被分配到多个阶段，可以并行执行不同阶段的计算任务，从而更好地利用流水线并行的优势。

混合并行HP

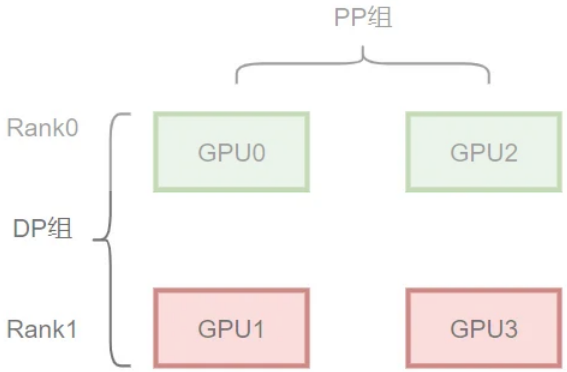
在进行上百亿/千亿级以上参数规模的超大模型预训练时，通常会组合上述(数据并行、张量并行、流水线并行)多种并行技术一起使用。常见的分布式并行技术组合方案如下。

DP+PP



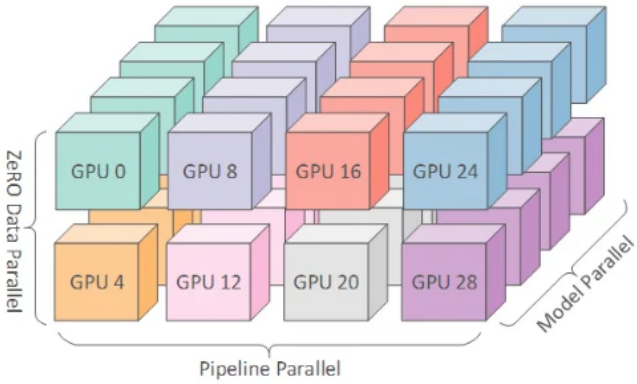
DP rank 0看不到GPU2， DP rank 1看不到GPU3，对于DP，只有GPU 0和1提供数据，就好像只有2个GPU一样。GPU0使用PP将其部分负载卸载到GPU2，同样的GPU1使用PP将其部分负载卸载到GPU3。

由于每个维度至少需要2个GPU，因此这里至少需要4个GPU。



3D并行(DP+PP+TP)

3D并行是由数据并行(DP)、张量并行(TP)和流水线并行(PP)组成。由于每个维度至少需要2个 GPU，因此这里至少需要8个GPU。



> **分布式训练并行策略选择**

单节点并行化策略

1. 当单个GPU可以装入整个模型时
   - DDP (Distributed DataParallel)
   - ZeRO
2. 当单个GPU无法装入整个模型时
   - Pipeline Parallel (PP)
   - Tensor Parallel (TP)
   - ZeRO
3. 当单个GPU无法装入模型的最大层时
   - 使用Tensor Parallel（TP），因为仅靠PipelineParallel（PP）不足以容纳大型层。
   - ZeRO

多节点并行化策略

1. 具有快速的节点间连接时
    - ZeRO
    - 组合使用PP、TP和DP
2. 节点间连接速度较慢且GPU内存不足时
    - 组合使用PP、TP、DP和ZeRO

LLM应用落地 6　　大模型 19

LLM应用落地 · 目录

上一篇
为什么大模型微调SFT之后还需要RLHF?

下一篇
零基础搞懂深度学习模型精度问题(FP32/FP16/BF16/Int8)!

修改于2024年07月30日